VELOCITE OPS / FICHE 01

# Remonter VéloCité

Le chemin rapide pour retrouver un projet utilisable, sans perdre les données.

Guide du 23 octobre 2026, fondé sur le dépôt à la révision **642a380**. Commandes PowerShell, depuis la racine du dépôt. Sous macOS/Linux, utiliser PowerShell 7 (**pwsh**).

| Votre situation | Chemin à suivre |
|---|---|
| Docker vient de redémarrer ; le cluster et ses données existent. | Préparation ci-dessous, puis reprise rapide. |
| Nouveau poste ou cluster absent. | Préparation ci-dessous, puis montage complet en page 2. |
| Base perdue ou données à retrouver. | Remonter les services, puis restauration en page 4. |
| Démonstration chez Hetzner. | Variante cloud en page 5 ; différente du montage local complet. |

## 1. Préparer le terminal

```
git pull --ff-only origin main
# Démarrer Docker Desktop, puis vérifier le moteur.
docker info --format '{{.ServerVersion}} {{.CgroupVersion}}'
$env:TF_VAR_phrase_etat = Read-Host 'Phrase du state'
```

Utiliser **la même phrase que celle des states existants**, au moins 16 caractères. Le moteur Docker doit répondre et annoncer cgroup **2**. Si Git refuse le pull, préserver le travail local avant de résoudre le conflit.

Sur un poste neuf : installer Docker Desktop et Git, puis lancer **./scripts/installer-outils.ps1**. Fermer et rouvrir PowerShell pour retrouver kind, tofu, kubectl et argo. Prévoir 8 Go de mémoire pour Docker au minimum, 12 Go conseillés.

## 2. Reprise rapide si le cluster et dev/prod existent

```
kubectl config use-context kind-velocite
kubectl get nodes
kubectl get namespaces
kubectl apply -k deploy/overlays/dev
kubectl apply -k deploy/overlays/prod
kubectl -n velocite-dev apply -f workflows
./scripts/smoke.ps1 -Env dev
./scripts/smoke.ps1 -Env prod
```

Si les nodes sont indisponibles, si un namespace manque ou si les contrôles échouent : passer à la page 2. Si un service démarre encore, attendre sa disponibilité avant de relancer les smoke tests.

# Montage local complet

Cluster kind, images, outils, deux environnements, applications et bilan.

**Point important :** dans cette version, up.ps1 ne termine pas le déploiement des applications et des workflows. Les étapes 2 et 3 ci-dessous sont donc indispensables. Son option -Controle n'est pas encore implémentée.

## 1. Créer ou réconcilier les fondations

```
# Si Compose tourne encore, le stopper sans effacer les volumes.
docker compose -f app/compose.yaml down
./scripts/up.ps1 -Env dev
./scripts/up.ps1 -Env prod -SansImages
```

Le premier passage construit toutes les images puis les charge dans kind ; le second réutilise ces images. Les outils communs sont réappliqués et les workspaces dev/prod créés ou sélectionnés. Les fichiers secrets de Compose ne sont pas nécessaires aux secrets créés par OpenTofu.

**Profil léger :** ajouter -Leger aux deux appels de up.ps1 seulement si ce profil est déjà celui du cluster, ou sur une nouvelle installation limitée en mémoire. Il crée un seul nœud et désactive Loki/Alloy. Changer le nombre de workers d'un cluster existant peut le recréer : sauvegarder d'abord.

## 2. Déployer les applications et attendre leur disponibilité

```
foreach ($e in @('dev', 'prod')) {
    $ns = "velocite-$e"
    kubectl apply -k "deploy/overlays/$e"
    foreach ($s in @('postgres', 'garage')) {
        kubectl -n $ns rollout status "statefulset/$s" `
            --timeout=300s
    }
    kubectl -n $ns wait --for=condition=complete `
        job/initialisation --timeout=600s
    foreach ($d in @('redis', 'api', 'ingest', 'nginx', 'loadgen')) {
        kubectl -n $ns rollout status "deployment/$d" `
            --timeout=300s
    }
}
```

Résultat attendu : PostgreSQL et Garage prêts, Job initialisation terminé, cinq déploiements disponibles. Si une commande échoue, arrêter la séquence et diagnostiquer avant de poursuivre.

## 3. Installer le bilan dans dev, puis tester

```
kubectl -n velocite-dev apply -f workflows
./scripts/smoke.ps1 -Env dev
./scripts/smoke.ps1 -Env prod
```

Le CronWorkflow programme le bilan à 02:30, heure de Paris. Le script de montage doit garder le même profil à chaque passage.

# Vérifier et accéder

Le projet est prêt lorsque les services répondent et que les contrôles passent.

## 1. Vérifications de base

```
kubectl config current-context
kubectl get nodes
kubectl -n velocite-dev get pods,job
kubectl -n velocite-prod get pods,job,hpa,pdb
./scripts/verifier.ps1 -Atelier 3,4,5,6,7,8,9
./scripts/promtool.ps1 test rules observability/tests/alertes_test.yaml
./scripts/promtool.ps1 test rules observability/tests/slo_test.yaml
```

Attendu : contexte kind-velocite, nodes Ready, services prêts, initialisation Complete, tests Prometheus verts. Les vérifications d'atelier incluent aussi des preuves documentaires et des exercices historiques : un KO doit être lu, même si les smoke tests passent.

| Service | Accès local |
|---|---|
| Application dev | https://localhost:8443 |
| Application prod | https://localhost:9443 |
| Grafana | http://localhost:3000 après le port-forward ci-dessous |
| Prometheus | http://localhost:9090 après le port-forward |
| Argo Workflows | http://localhost:2746 après le port-forward |

## 2. Ouvrir les outils : un terminal par port-forward

```
kubectl -n observabilite port-forward svc/grafana 3000:80
# Dans un autre terminal :
kubectl -n observabilite port-forward svc/prometheus-server 9090:80
# Dans un troisième terminal :
kubectl -n argo port-forward svc/argo-workflows-server 2746:2746
```

Les commandes restent au premier plan ; Ctrl+C ferme le tunnel. Compte Grafana : admin. Lire son mot de passe dans le terminal avec **tofu -chdir=infra/10-outils output -raw grafana_mot_de_passe**. Ne pas le copier dans Git ou une capture partagée.

Les frontaux utilisent une autorité de certification interne. Les smoke tests récupèrent cette autorité et vérifient TLS ; un navigateur peut afficher un avertissement tant que cette autorité n'est pas reconnue localement.

## 3. Vérifier ou rejouer le bilan

```
argo -n velocite-dev cron list
argo -n velocite-dev list
# Essai sur une date du jeu de démonstration :
argo -n velocite-dev submit `
    --from workflowtemplate/bilan-quotidien -p date=2026-03-12
argo -n velocite-dev get '@latest'
./scripts/compter-bilan.ps1 -Date 2026-03-12 -Env dev
```

Le workflow doit finir en Succeeded. Relire son état après la fin ; une seconde exécution pour le même jour ne doit pas dupliquer le bilan.

# Sauvegarder et restaurer

Remonter les services ne suffit pas à récupérer les données perdues.

## 1. Avant une destruction : sauvegarder chaque base

```
kubectl config use-context kind-velocite
./scripts/sauvegarder.ps1 -Env dev
./scripts/sauvegarder.ps1 -Env prod
Get-ChildItem sauvegardes/*.sql | Select-Object Name, Length
```

Les exports SQL doivent être non vides et copiés dans un emplacement sûr hors du dépôt. **Ils ne contiennent ni les objets S3 de Garage ni les states OpenTofu.** Préserver également ces éléments et la phrase du state selon le besoin de reprise. Un clone Git seul ne récupère pas les anciennes données.

## 2. Restaurer dev, une fois les services remontés

Cette opération remplace les tables de la base choisie. Suspendre les bilans et attendre qu'aucun workflow n'écrive encore ; puis arrêter les services qui produisent des écritures.

```
argo -n velocite-dev cron suspend bilan-quotidien
argo -n velocite-dev list
# Continuer seulement après la fin des workflows en cours.
kubectl -n velocite-dev scale deployment/api `
    deployment/ingest deployment/loadgen --replicas=0
./scripts/restaurer.ps1 -Env dev
kubectl apply -k deploy/overlays/dev
kubectl -n velocite-dev rollout status deployment/api --timeout=300s
kubectl -n velocite-dev rollout status deployment/ingest --timeout=300s
kubectl -n velocite-dev rollout status deployment/loadgen --timeout=300s
./scripts/smoke.ps1 -Env dev
argo -n velocite-dev cron resume bilan-quotidien
```

Sans -Fichier, restaurer.ps1 prend le dernier SQL dont le nom correspond à l'environnement. Pour imposer une sauvegarde, ajouter **-Fichier sauvegardes/NOM_DU_FICHIER.sql** en remplaçant le nom. Pour prod, utiliser -Env prod et le namespace velocite-prod ; le bilan fourni ici est installé seulement dans dev.

## 3. Preuve de l'atelier 10, si la sauvegarde la contient

```
./scripts/donnees-du-jour.ps1 -Verifier
./scripts/verifier.ps1 -Atelier 10
```

Attendu : **250 trajets**, empreinte **9b7904d63be2**. Ne pas relancer l'insertion après une panne pour fabriquer cette preuve : les lignes doivent provenir de la restauration. Le script affiche le RTO de restauration seule.

## Destruction volontaire du local

```
./scripts/down.ps1
```

À lancer uniquement après sauvegarde : détruit tout le cluster kind, ses volumes et les states locaux actifs des couches 10/20. Ce n'est pas une commande de redémarrage. Ne pas utiliser panne.ps1 pour une simple remise en route.

# Variante Hetzner

Le parcours cloud de l'atelier 11 déploie les applications dev sur k3s.

**Périmètre :** cloud.ps1 monte le serveur et applique infra/20-env puis l'overlay dev. Il n'installe pas la couche locale 10-outils ni les workflows. Pour retrouver toute la supervision et le bilan du projet, suivre le parcours local des pages 1 à 3.

## 1. Préparer les variables dans un même terminal

```
$s = Read-Host -AsSecureString 'Token Hetzner'
$env:HCLOUD_TOKEN = [Net.NetworkCredential]::new('', $s).Password
$env:TF_VAR_eleve = 'velocite-equipe-MAURY-PADONOU-ADNANE'
$env:TF_VAR_phrase_etat = Read-Host 'Phrase du state'
```

Conserver le token et la phrase hors de Git. Pour un state cloud existant, reprendre sa phrase initiale. Garder la même connexion réseau/VPN pendant toute l'opération : le pare-feu est limité à l'IP publique du poste en /32.

## 2. Monter, déployer, vérifier - dans cet ordre

```
./scripts/cloud.ps1 -Etape monter
./scripts/cloud.ps1 -Etape deployer
./scripts/cloud.ps1 -Etape verifier
```

Attendre la réussite de chaque commande avant la suivante. Lire le plan avant de répondre yes. Au premier montage, cinq ressources sont prévues. Lors d'une reprise, le contrôle autorise aussi la mise à jour du pare-feu ; il refuse les suppressions, remplacements et autres modifications.

monter crée le serveur, attend /var/lib/velocite-pret et ajoute le contexte kubectl hetzner-<équipe>. deployer construit les images sur la machine et attend les services. verifier appelle l'API en HTTPS avec le certificat vérifié. Le script de déploiement ne doit pas être rejoué aveuglément : il recrée sa copie locale .env-cloud, qui contient aussi son state.

## 3. Si l'attente de k3s expire

```
# Remplacer IP_DU_SERVEUR par la sortie du montage.
$cle = "infra/00-cloud-hetzner/.cles/$env:TF_VAR_eleve"
ssh -i $cle root@IP_DU_SERVEUR `
    'cloud-init status --long; tail -n 80 /var/log/cloud-init-output.log'
```

Une erreur SSH peut empêcher la vérification alors que k3s est déjà prêt. Si l'IP du poste a changé, relancer monter. Si SSH annonce une identité différente, vérifier l'IP et l'identité via Hetzner avant de remplacer l'entrée connue ; ne pas supprimer tout known_hosts.

## 4. Terminer la démonstration et revenir au local

```
./scripts/cloud.ps1 -Etape detruire
kubectl config use-context kind-velocite
```

Une machine simplement éteinte reste facturée. La destruction supprime aussi ses données : sauvegarder avant si nécessaire. Vérifier dans Hetzner ou via son API que le serveur a bien disparu.

# Garder une reprise propre

Quelques contrôles ciblés évitent les reconstructions inutiles.

| Symptôme | Vérifier et agir |
|---|---|
| Docker ou Kubernetes ne répond pas | Démarrer Docker Desktop ; vérifier docker info puis kubectl config current-context. Le contexte local est kind-velocite. |
| ImagePullBackOff / ErrImagePull | Après une recréation de kind, relancer ./scripts/charger-images.ps1. Les images velocite/* sont locales, pas publiées dans un registre. |
| Job initialisation en échec | Lire kubectl -n velocite-dev describe job initialisation et les logs du pod : conteneurs stockage et seed. Corriger la cause avant de recréer le Job. |
| Pods Pending | Lire describe pod et get events ; vérifier mémoire Docker, quotas et PVC. Le profil léger doit être choisi de façon cohérente dès le montage. |
| CrashLoopBackOff | Lire les logs du pod avec --previous puis les événements. Ne pas détruire le cluster pour masquer l'erreur. |
| Phrase du state incorrecte | Reprendre la phrase originale. Une nouvelle phrase ne déchiffre pas les states existants ; conserver les fichiers pour la reprise. |
| Service disponible mais vérificateur KO | Lire chaque contrôle : certains exigent un exercice antérieur, une donnée restaurée ou une section documentaire, pas seulement un pod prêt. |

## Prouver la cohérence sans appliquer de changements

```
tofu -chdir=infra/00-cluster plan -detailed-exitcode
tofu -chdir=infra/10-outils plan -detailed-exitcode
foreach ($e in @('dev', 'prod')) {
    tofu -chdir=infra/20-env workspace select $e
    tofu -chdir=infra/20-env plan -detailed-exitcode `
        "-var-file=envs/$e.tfvars"
    kubectl diff -k "deploy/overlays/$e"
}
```

Pour tofu plan : code 0 = aucun changement, 2 = changements proposés, 1 = erreur. Pour kubectl diff : 0 = aucun écart, 1 = différences. En profil léger, ajouter -var=nb_workers=0 au plan 00 et -var=leger=true au plan 10. Ne pas modifier les replicas d'API prod gérés par le HPA pour forcer un diff vide.

## Ce qui doit rester disponible hors de Git

Phrase du state, states OpenTofu et copies de sauvegarde, exports SQL, objets Garage nécessaires et, pour le cloud, token et clé SSH. Ces éléments doivent être protégés et conservés séparément. Ils ne figurent pas dans ce guide.

Références du dépôt : scripts/up.ps1, charger-images.ps1, smoke.ps1, sauvegarder.ps1, restaurer.ps1, down.ps1, cloud.ps1 ; deploy/overlays ; workflows ; infra/00-cluster, 10-outils et 20-env ; docs/RUNBOOK.md. Guide établi par lecture du code ; aucune destruction ni reconstruction de l'infrastructure n'a été exécutée pour produire ce PDF.